

CS 380: Math for AI; Homework #2: The Matrix

Ben Belden

October 7, 2026

Due Monday, October 5 at 11:59pm Central

Please read the following independent completion attestation before starting the homework. Then digitally sign it, acknowledging that you understand the guidelines for this homework, and have followed them.

I declare that I have completed this assignment entirely on my own,
and following the generative AI policies.

Googling for documentation and other basic material (Wikipedia Fibonacci number) is fine,
but no use of tools like ChatGPT, either to write code or to write text.

I have read the UAB Academic Integrity Code and understand that any breach
of this code may result in severe penalties, including failure of the class.

Signature: *Ben Belden*

Symbols Reference

Here are some symbols that may be useful for this problem set
(see the corresponding L^AT_EX in the raw document):

- summation and inner product: $a \cdot b = \sum_{i=1}^n a_i * b_i$
- norm: $\|v\|_2$
- fraction: $\frac{1}{2}$
- θ , as in $\cos \theta = a \cdot b$ (when both unit)
- vector: $\begin{pmatrix} 0 & 1 \end{pmatrix}$
- matrix:

$$\begin{pmatrix} 0 & 1 & 2 \\ 3 & 4 & 5 \end{pmatrix}$$

- for pretty printing, you may want to add a newline using backslash backslash (see Q1 for an example)
- to force a new page to be begun (especially if LaTeX is acting strangely at the end of a page), use the newpage command (see a few lines below)

L^AT_EX typing tips:

- Exponents (use curly braces if exponent is more than 1 character): x^2 , 2^{3x}
- Subscripts (use curly braces if subscript is more than 1 character): x_0 , x_{10}
- **Boldface** and *italics*

All written answers will be added to this TeX document.

Replace all 'Write your answer here ...' by your answer, removing the 'Write your answer here ...'.

Q1: Data Matrix (code)

In this question, you will build the data matrix of a point cloud from a PLY file and from a PDB file. This will help gain you experience in how to read data, as well as exposing you to PLY (an important data format) and to the Protein Data Bank (an important resource for computational structural biology and AI for science, such as the recent AlphaFold Nobel Prize).

Write a function to read a point cloud in PLY format into a column-major data matrix, using the x/y/z properties of the vertex element. The output will be a matrix of shape (3, n). Use this function to extract the 5 points of the Stanford bunny starting at 30,000th point and write this submatrix to 'bunny.X'

Stanford Bunny Submatrix:

to download:

```
curl -O https://graphics.stanford.edu/pub/3Dscanrep/bunny.tar.gz
```

```
tar -xzf bunny.tar.gz
```

dir bunny consists of sub-directories → data, reconstruction:

data - different scans, views of the bunny

reconstruction - reconstructed, resampled versions of the bunny

How do we find the location of the 30,000, 30,001, 30,002, 30,003, 30,004 points?

1) check the number of points total used to assemble the bunny

```
head -20 bun_zipper.ply      element vertex 35947
```

2) extract from bun_zipper.ply

NOTE: each vertex line in bun_zipper.ply is structured as so: x, y, z, confidence, intensity (5)

Points, Stanford Bunny:

$$\begin{pmatrix} -0.0077906 & -0.00878734 & -0.00979029 & -0.0107915 & -0.0117867 \\ 0.079881 & 0.0798675 & 0.0798633 & 0.0798668 & 0.0798822 \\ -0.0380243 & -0.0379507 & -0.0379189 & -0.0380291 & -0.0383021 \end{pmatrix}$$

Write a function to read the atoms of a PDB format into a column-major data matrix, using the ATOM and HETATM lines (lec09). Use this function to extract the 5 points of 1GCN.pdb starting at 100th point and write this submatrix to '1GCN.X'

I am staring at glucagon ...

'1GCN.pdb' submatrix →

ATOM sub-matrix:				
100	101	102	103	104
CB	CG	CD1	CD2	CE1
TYR	TYR	TYR	TYR	TYR
A	A	A	A	A
13	13	13	13	13
27.73	26.516	25.798	26.185	24.713
23.578	24.5	24.377	25.498	25.228
13.385	13.692	14.867	12.796	15.12
1.0	1.0	1.0	1.0	1.0
8.0	8.0	8.0	8.0	8.0
C	C	C	C	C

Resources:

- Stanford 3D Scan Repository (see lec08) for Stanford bunny
- Greg Turk's documentation of the PLY format ([turk94ply.pdf](#))
- Paul Bourke's documentation: paulbourke.net/dataformats/ply
- documentation of the plyfile package (see lec08)
- KITTI-360 cvlibs.net/datasets/kitti-360/documentation.php (3D Data Format); another source of PLY files, with richer properties
- Protein Data Bank (PDB): <https://www.rcsb.org/> and UCSF tutorial: <https://www.cgl.ucsf.edu/chimera/docs/UsersGuide/tutorials/pdbintro.html> (lec09)



Figure 1: Jeff.

Q2: Mean Normalization (code)

In this question, you will mean-normalize a point cloud, such as the point clouds you just generated. Write a function for the mean of a point cloud, without using NumPy.

→ because the Stanford bunny consists of so many x, y, z vertices (35947) and faces connecting these points in 3-space (69451), checking the calculated mean values of each coordinate is tedious and long.

To confirm correctness, I used 'print' statements → printing the first x, y, z vertex triple to stdout for check, the last x, y, z vertex triple to stdout for check, and the first line corresponding to a 'property face'.

Although this does not confirm correct calculations, it is enough information to allow me to proceed with confidence.

bunny μ :

$$(-0.02675991 \quad 0.09521606 \quad 0.00894711)$$

Write a function that computes the standard deviation of each coordinate of a point cloud, without using NumPy.

jeff the bunny is a peculiar little fellow - jeff's standard deviation, σ :

$$\begin{bmatrix} 0.0409879 \\ 0.04153102 \\ 0.02816427 \end{bmatrix}$$

Write a function for matrix multiplication, without using NumPy, that is using a triple 'for' loop. Then use this function to write a function to compute the matrix transform that mean normalizes a given point cloud in d -space. The shape of this matrix will be (4, 4), because of the translation involved in mean normalization.

jeff is on the move towards, how is he going to move towards this destination???

T :

$$\begin{bmatrix} 1 & 0 & 0 & 0.02675991 \\ 0 & 1 & 0 & -0.09521606 \\ 0 & 0 & 1 & -0.00894711 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Write a function to build a mean-normalized cloud, by applying the transform of the previous function to a point cloud. You should try to vectorize this code. That is, avoid multiplying each point separately, but instead multiply all points at the same time.

To test your code, note that the mean of the mean-normalized point cloud should be the origin and its standard deviation vector should be (1, ..., 1).

jeff is currently not satisfied with his current location, the origin seems more suitable for a rabbit such as jeff

$[P_{ij}]...$

```
-0.011069790003039818  0.0327239401813781  -0.004472444579289702
-0.018019490003039814  0.0336709401813781  -0.007042144579289702
-0.04124959000303982  0.05602794018137809  0.0282481854207103
 0.024472499996960183  0.03493394018137809  0.014272985420710298
 0.004154509996960182  0.03145894018137811  -0.0017912445792897021
```

NOTE: because rounding is quite after the decimal (.), each mean normalized point is a row vector rather than a column vector

$(x_1, y_1, z_1) = \text{row 1, etc. etc. etc.}$



Figure 2: Jeff.

Q3: Matrix Multiplication (code)

In this question, you will implement matrix multiplication yourself, using a triple for loop, and compare the speed of this implementation with the built-in NumPy matrix multiplication.

Lesson 1: it is important to understand an operation by implementing it yourself.

Lesson 2: after you understand it, it is important to use the NumPy function, since it is much faster.

Write a function for matrix multiplication yourself, without using NumPy. Use a triple for loop (three nested for loops). Only use NumPy to store the matrices, but not to compute the multiplication. As usual, I have provided you with template code.

Build 4 pairs of random matrices: 2 each of shape (32, 32), (64, 64), (128, 128), and (512, 512). Time your version of matrix multiplication on these pairs, then time the native NumPy matrix multiplication on these pairs. Also test that your code is correct by comparing the two answers.

Report your timings in the table below.

Method	32x32	64x64	128x128	512x512
Yours	0.0164435 sec	0.0992624 sec	0.8332436 sec	43.5817902 sec
NumPy	0.0002637 sec	0.0001058 sec	0.0004239 sec	0.0106391 sec

NumPy code is much faster because it is written in compiled C and C++ code under the hood (maybe I should switch to C++, Eigen setup, I don't know). It stores data in consecutive memory locations making it easier to manipulate and 'grab'.

My implementation (triple 'for' loop) is slow. Why? The data structure under the hood is a Python list of lists (rows, cols). Python stores this as an array of pointers to other arrays (whew). The pointers in our original list point to our 'index' arrays in a completely separate location in memory, in which this array pointed to points to float objects located far and wide in RAM. The CPU has to constantly search for data from a massive area to search from (whew).

CPU is fast at performing calculations (ALU), but slow at pulling data from RAM.

P.S. -> my implementation, interpreter ...

Approximately how much faster is it on the (512, 512) matrices? $\text{Speedup} = \text{Slower} / \text{Faster}$

– 4,096 times faster

To code time accurately (in Python), you use the 'time' module. You measure the time before the code (function) runs and after the code (function) runs. You then subtract the end time from the start time resulting in total allotted time while a particular code block is running.

NumPy function that you use to test whether two matrices are the same: `np.array_equal`

Does this function use equality tests? Yes

Why? to make sure precise equality between the two matrices, to account for weird floating-point rounding situations as well

Q4: Fibonacci Matrix (proof and code)

Let

$$F = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}$$

Then

$$F^n = \begin{pmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{pmatrix}$$

Proof.

Induction

1) Base Case: $n = 1, n = 2, n = 3$

$n = 1;$

$$F_{n+1} = F_2 = 1, F_n = F_1 = 1, F_{n-1} = F_0 = 0$$

$$F_{n+1} = F_n + F_{n-1}$$

$$F_{n+1} = 1 + 0$$

$$F_{n+1} = F_2 = 1$$

$F \rightarrow$

$$\begin{pmatrix} F_2 & F_1 \\ F_1 & F_0 \end{pmatrix}$$

$n = 2;$

$$F_{n+1} = F_3 = 2, F_n = F_2 = 1, F_{n-1} = F_1 = 1$$

$$F_{n+1} = F_n + F_{n-1}$$

$$F_{n+1} = 1 + 1$$

$$F_{n+1} = F_3 = 2$$

$n = 3;$

$$F_{n+1} = F_4 = 3, F_n = F_3 = 2, F_{n-1} = F_2 = 1$$

$$F_{n+1} = F_n + F_{n-1}$$

$$F_{n+1} = 2 + 1$$

$$F_{n+1} = F_4 = 3$$

$n = 1 \rightarrow$

$$\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}$$

$n = 2 \rightarrow$

$$\begin{pmatrix} 2 & 1 \\ 1 & 1 \end{pmatrix}$$

$n = 3 \rightarrow$

$$\begin{pmatrix} 3 & 2 \\ 2 & 1 \end{pmatrix}$$

2) Inductive Hypothesis: $n = k$

Assume that:

$F^k =$ is true $\rightarrow$

$$\begin{pmatrix} F_{k+1} & F_k \\ F_k & F_{k-1} \end{pmatrix}$$

Base values: $k = 0 : 0, k = 1 : 1$

$$k = 2 : (F_{k-1} = F_1 = 1) + (F_{k-2} = F_0 = 0) = 1$$

$$k = 3 : (F_{k-1} = F_2 = 1) + (F_{k-2} = F_1 = 1) = 2$$

$$0, 1, 1, 2, 3, \dots, k = (F_{k-1}) + (F_{k-2})$$

3) Inductive Step: $n = (k + 1)$

$$F^{k+1} = \rightarrow$$

$$\begin{pmatrix} F_{(k+1)+1} & F_{k+1} \\ F_{k+1} & F_{(k+1)-1} \end{pmatrix}$$

$$\begin{pmatrix} F_{k+2} & F_{k+1} \\ F_{k+1} & F_k \end{pmatrix}$$

$$0, 1, 1, 2, 3, \dots, F_k = (F_{k-1} + F_{k-2}), F_k + 1 = (F_{k-1} + F_{k-2}) + 1$$

$$F_2 = 1 \rightarrow F_1 + F_0, F_4 = 3 \rightarrow F_3 + F_2, F_3 = 2 \rightarrow 1 + 1$$

$$F_k \rightarrow F_{k-1} + F_{k-2}, F_{k+2} \rightarrow F_{k+1} + F_k, F_{k+1} \rightarrow F_k + F_{k-1}$$

$$\begin{pmatrix} F_{k+1} + F_k & F_k + F_{k-1} \\ F_k + F_{k-1} & F_{k-1} + F_{k-2} \end{pmatrix}$$

$$F^{k+1} = F^k * F(?) =$$

$$\begin{pmatrix} F_{k+1} & F_k \\ F_k & F_{k-1} \end{pmatrix} \times \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} F_{k+1} + F_k & F_{k+1} \\ F_k + F_{k-1} & F_k \end{pmatrix}$$

$$\begin{pmatrix} F_{k+1} + F_k & F_{k+1} \\ F_k + F_{k-1} & F_k \end{pmatrix} = \begin{pmatrix} F_{k+2} & F_{k+1} \\ F_{k+1} & F_k \end{pmatrix}$$

$$F^k \text{ true} \rightarrow F^{k+1} \text{ true}$$

✓
■

Implement a logarithmic-time algorithm to compute the nth Fibonacci number, using matrix multiplication. One of the key observations is above. Hint: do not use brute force iteration over matrix multiplication, since that would yield a linear algorithm.

For the logarithmic Fibonacci algorithm implementation:

Time taken to compute F_1000 is 0.00012135505676269531 sec

Um, this algorithm took myself some time to wrap my head around.

I had to come to the reality that both F_0 and F_1 do not take any influence from previous (or future) "patterns, sequences, etc." to compute their respected Fibonacci numbers 0, 1. Starting somewhere is required

I deduced that the matrix F from the proof must be a representation of F_1 as all entries F_{n+1}, F_n, F_{n-1} correspond to the correct Fibonacci number of $F_{0,1,2}$ represented in F . The match presents itself with one of the main realizations one must have to grasp what is going on $\rightarrow F_{n+1} = F_n + F_{n-1}$. From here I checked to see if this pattern within F^n , F continues as you increase 'n', it does.

As multiplication of the matrix portrayed by F and a current fib state $[F_n \ F_{n-1}]^T$ repeatedly happens, each output traverses through the Fibonacci numbers $\rightarrow [F_{n+1} \ F_n]^T$, so the choice of F is not random whatsoever.

As you multiply current states by F , you have the ability to find future Fibonacci numbers, so if you went through the process of repeatedly multiplying a current state by F you would eventually find the Fibonacci number you are looking for. This continual multiplication results in a linear time complexity, the question asks for an algorithm that can find the Fibonacci number you are looking for in $O(\log n)$ time.

The matrix fib algorithm works by recursively splitting the 'fibNo' in half until a base case of $n == 0$ is reached in which the identity matrix is returned. Upon arrival at the base case, the algorithm moves back

up the stack frame performing matrix multiplications (starting with base case '1'). A multiplication depends on whether the current frames 'fibNo' is either even or odd. The key word in this subtle explanation is HALF.

Because the 'fibNo' is cut in half during each recursive step, it take $O(\log n)$ time $\rightarrow$ binary exponentiation.

What is the largest Fibonacci number you can compute using NumPy scalar types, before it overflows?

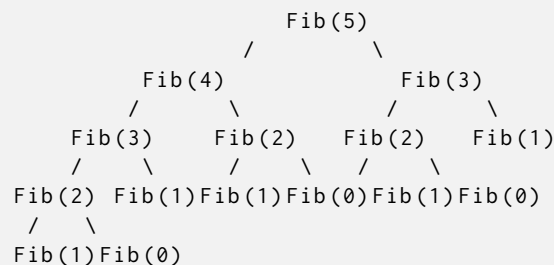
The largest Fibonacci number you can compute using NumPy scalar types before overflowing depends on whether you use a signed or unsigned 64-bit integer.

The largest exact Fibonacci number you can compute is F_{93} , which equals 12,200,160,415,121,876,738. If you use a signed 64-bit integer, the limit is F_{92} , which equals 7,540,113,804,746,346,429.

NumPy scalar types use C-style data types so memory size is fixed. As you reach the 'top' of a 64 bit float, precision begins to diminish. To confirm precision and memory capability this threshold is set. Over this threshold, bits begin to shift and 'rounding' of some sort begins. (I think)

Recursion is usually a wonderful approach to problems. However, in the case of Fibonacci numbers, it leads to an exponential algorithm.

each function call branches into two more calls, resulting in calculations of the same subproblems calculated previously, super redundant $\rightarrow$ to calculate a number such as Fib(5):



Fib(5) requires 15 total function calls.

Ungraded challenge

Prove that affine combination of points is legal, yet, in general, addition of points is undefined.
(After further reflection, I thought that three proofs in this HW would be too much.)

→ although this sounds super interesting, I will bypass as I am tired. Affine combinations are cool though.

What to hand in

Generate a pdf of this document using Overleaf, naming the file 'hw2-LASTNAME-26fa380.pdf'.

Example: hw2-johnstone-26fa380.pdf

Write your code in a Python script named 'hw2-LASTNAME-26fa380.py'.

Example: hw2-johnstone-26fa380.py

I have given you a starting point in template-hw2-26fa380.py.

Please use this code as a starting point (e.g., use the same function names).

Zip the pdf and Python script into hw2-LASTNAME-26fa380.zip, and upload this one zip file to Canvas.